

YUTrade - Final Project Report

Project Name: YUTrade - York University Campus Marketplace
Course: EECS 4314 - Advanced Software Engineering (Winter 2026)
Date: April 5, 2026
Repository: <https://github.com/mai2161/YUTrade>
Deployment: Vercel + Render

Team Members

Name	Primary Contribution
Daniel Chahine	Authentication, JWT security, password flows
Michael Byalsky	Database models, ORM, fixtures
Lakshan Kandeepan	Listings CRUD, images, filters
Rajendra Brahmabhatt	Messaging, threads, CI/CD
Mai Komar	Frontend pages, API integration, routing
Harnaindeep Kaur	Frontend components, styling, auth context

1. Demo Access Instructions

- YouTube walkthrough:** <https://youtu.be/mwINZk4j2iw>
- Live frontend (Vercel):** <https://yu-trade.vercel.app/>
- Live backend API (Render):** <https://yutrade-api.onrender.com>

Evaluation:

- Open the frontend URL and register using a YorkU email format (`@my.yorku.ca` or `@yorku.ca`)
- Browse listings, apply search/filters, open a listing, and test messaging/rating flows
- API endpoints can be checked directly using the backend URL (or `/docs` if enabled in deployment)

2. Requirements Traceability Matrix - Functional Requirements (/6)

Req ID	Requirement	Status	Key Evidence
FR-01	Register with YorkU email validation	Met	schemas/auth.py, services/auth_service.py
FR-02	Login with JWT issuance	Met	routers/auth.py: login, utils/security.py
FR-03	Forgot/reset password (6-digit code, expiry, one-time use)	Met	services/auth_service.py
FR-04	Profile/account management (/auth/me,	Met	routers/auth.py, services/auth_service.py

	change password, delete account)		
FR-05	Create listing with optional image upload	Met	routers/listings.py:create_listing, services/listing_service.py
FR-06	Browse listings with pagination/search/filter/sort	Met	services/listing_service.py:get_listings
FR-07	View single listing	Met	routers/listings.py:get_listing_by_id
FR-08	Update/delete listing (owner-only, image add/delete, cascade cleanup)	Met	services/listing_service.py:update_listing/delete_listing
FR-09	Buyer-seller messaging and seller reply flow	Met	routers/messages.py, services/message_service.py
FR-10	Read tracking + thread listing	Met	routers/messages.py, routers/threads.py
FR-11	Rating workflow (eligibility, create/update/delete, seller averages)	Met	routers/ratings.py, services/rating_service.py
FR-12	Frontend feature coverage (auth, listings, messages, ratings, protected routes)	Met	frontend/src/pages/*, frontend/src/components/ProtectedRoute.tsx

RTM summary: Functional requirements are fully implemented and traced to concrete code + tests without listing all 40 line items in full.

3. Requirements Traceability Matrix - Non-Functional Requirements (/4)

NFR ID	Category	Requirement	Status	Evidence
NFR-01	Security	JWT-based auth and bcrypt password hashing	Met	backend/app/utils/security.py
NFR-02	Security	Authorization checks for protected/owner-only actions	Met	dependencies.py, listing/message/rating service checks
NFR-03	Validation	Strong server-side input validation with typed schemas	Met	backend/app/schemas/*.py

NFR-04	Performance	Pagination and bounded query limits for listing endpoints	Met	routers/listings.py query params (page, limit)
NFR-05	Maintainability	Layered architecture (Router -> Service -> Data)	Met	backend/app/routers, backend/app/services, ORM models
NFR-06	Configuration	Env-driven settings and deploy-ready config	Met	config.py, .env.example, render.yaml
NFR-07	UI Responsiveness	Mobile-friendly responsive UI and consistent design tokens	Met	frontend/src/styles/global.css, variables.css, Navbar.tsx
NFR-08	Accessibility	Focus states/semantic tags/basic ARIA labels	Partially Met	Good baseline, but broader ARIA/test coverage still limited

4. Architecture Diagram (/5)

```
graph TB
    Browser[Browser] --> React[React + TypeScript SPA]
    React --> API[FastAPI Backend]

    API --> AuthR[Auth Router]
    API --> ListR[Listings Router]
    API --> MsgR[Messages/Threads Routers]
    API --> RateR[Ratings Router]

    AuthR --> AuthS[Auth Service]
    ListR --> ListS[Listing Service]
    MsgR --> MsgS[Message Service]
    RateR --> RateS[Rating Service]

    AuthS --> ORM[SQLAlchemy Models]
    ListS --> ORM
    MsgS --> ORM
    RateS --> ORM

    ORM --> DB[(SQLite)]
    ListS --> Uploads[(Local uploads)]

    AuthS --> Email[Email Service]
    Email --> Resend[Resend/SMTP]
```

Summary: YUTrade uses a 3-tier architecture: React frontend, FastAPI service layer, and SQLite data layer via SQLAlchemy.

5. Sequence Diagrams (/3)

5.1 Registration + Login

```
sequenceDiagram
    actor User
    participant FE as React
    participant API as FastAPI
    participant Auth as Auth Service
    participant DB as SQLite

    User->>FE: Submit register form
    FE->>API: POST /auth/register
    API->>Auth: register_user()
    Auth->>DB: Insert user (hashed password)
    API-->>FE: 201 Created

    User->>FE: Submit login form
    FE->>API: POST /auth/login
    API->>Auth: authenticate_user()
    Auth->>DB: Validate credentials
    API-->>FE: JWT + user
```

5.2 Listing + Messaging

```
sequenceDiagram
    actor Seller
    actor Buyer
    participant FE as React
    participant API as FastAPI
    participant LS as Listing Service
    participant MS as Message Service
    participant DB as SQLite

    Seller->>FE: Create listing + images
    FE->>API: POST /listings
    API->>LS: create_listing()
    LS->>DB: Save listing + image metadata

    Buyer->>FE: Message seller
    FE->>API: POST /listings/{id}/messages
    API->>MS: send_message()
    MS->>DB: Insert message
```

5.3 Rating Flow

```
sequenceDiagram
    actor Buyer
    participant FE as React
    participant API as FastAPI
    participant RS as Rating Service
    participant DB as SQLite

    FE->>API: GET /listings/{id}/rating/me
    API->>RS: check eligibility
    RS->>DB: Verify prior messaging

    Buyer->>FE: Submit score/comment
    FE->>API: POST /listings/{id}/rating
    API->>RS: create_rating()
    RS->>DB: Insert rating
```

6. Class Diagram / OOP Justification (/2)

```
classDiagram
    class User
    class Listing
    class Image
    class Message
    class Rating
    class VerificationCode
    class PasswordResetCode

    User "1" --> "*" Listing
    Listing "1" --> "*" Image
    Listing "1" --> "*" Message
    Listing "1" --> "*" Rating
    User "1" --> "*" Message
    User "1" --> "*" Rating
```

Justification:

- Backend uses OOP through SQLAlchemy model classes and relationships.
- Frontend follows modern React functional composition (components + hooks + typed interfaces).

7. Design Patterns & Course Concepts (/5)

Design patterns used:

- Repository-style services (`services/*.py`)
- Dependency Injection (`Depends(get_db)` , `Depends(get_current_user)`)
- Middleware (CORS)
- Provider/Guard patterns in frontend (`AuthContext` , `ProtectedRoute`)
- Strategy pattern for email backends (`console/Resend/SMTP`)

Course concepts demonstrated:

- RESTful API design and status codes
- Separation of concerns (frontend modules + backend layering)
- DRY via shared helpers/fixtures
- SOLID (especially SRP and DIP)
- Validation on client + server

8. Functional Requirements Demonstration (/5)

Requirement	Expected	Actual
Registration/login	Valid YorkU user gets account + JWT	Works; invalid domain/password rejected
Password reset	Code-based reset flow	Works with expiry + one-time use
Listing CRUD	Owner can create/edit/delete with images	Works; owner checks enforced
Browse/search/filter	Paginated searchable listings	Works with sort and price/category filters
Messaging	Buyer-seller communication per listing	Works with threads and read tracking
Ratings	Eligible buyers can rate sellers	Works; duplicate/invalid ratings blocked

9. Usability & UI Consistency (/5)

- Shared design system in `variables.css` (colors, spacing, typography, shadows)
- Responsive layout and mobile navigation
- Consistent navigation and auth-aware route behavior
- Clear loading/error handling on major pages
- Accessibility baseline present (`:focus-visible` , semantic tags, basic ARIA)

Known gap: broader ARIA coverage and automated accessibility testing are still needed.

10. Source Code Repository & Good SENG Practices (/5)

- Clear repository structure (`backend` , `frontend` , `docs` , CI config)
- Environment examples (`backend/.env.example` , `frontend/.env.example`)
- `.gitignore` includes secrets, generated files, and local artifacts
- Strong team collaboration (feature branches + PR workflow)
- Deployment config present (`render.yaml`)
- Automated workflow integration in `.github/workflows`

11. Test Cases Tied to RTM (/5)

Instead of listing all 133 entries here, tests are grouped and traced as follows:

Test Range	Focus
TC-01 to TC-44	Authentication/account lifecycle
TC-45 to TC-83	Listings + browse + image handling
TC-84 to TC-105	Messaging + thread behavior
TC-106 to TC-130	Ratings + eligibility + averages
TC-131 to TC-133	End-to-end integration flows

All groups have direct requirement links in the test suite documentation.

12. Unit Test Cases & Data (/5)

Backend test suite: 254 tests across 9 files

File	Count	Scope
test_auth.py + test_auth_extended.py	74	Auth, profile, passwords, edge cases
test_listings.py + test_listings_extended.py	82	CRUD, images, filters, pagination
test_messages.py + test_messages_extended.py	38	Messaging, threads, read state
test_ratings.py + test_ratings_extended.py	53	Rating lifecycle and constraints
test_integration.py	7	End-to-end user journeys

Infrastructure: pytest + FastAPI TestClient + in-memory SQLite fixtures.

Gap: frontend automated tests are not yet implemented.

13. Test Case Outcomes (/5)

Execution command: `cd backend && python -m pytest tests/ --tb=short`

Result: 254 passed, 0 failed

Duration: ~122 seconds

Category	Outcome
Authentication tests	Pass
Listings tests	Pass
Messaging tests	Pass
Ratings tests	Pass
End-to-end tests	Pass

Non-blocking deprecation warnings were observed (datetime/Pydantic/passlib), but no functional failures.

14. Summary & Self-Assessment

Criterion	Max	Self-Score	Notes
Functional RTM	6	5.5	Strong traceability; frontend test IDs are less explicit
Non-functional RTM	4	3.5	Accessibility partly complete
Architecture + sequence + class diagrams	10	10	Diagrams and explanations included
Patterns + concepts	5	5	Clear links to implementation
Functional demo + usability	10	8.5	Features work; accessibility can improve
Repo practices	5	5	Strong collaboration + structure + deployment
Testing evidence + outcomes	15	14	254/254 backend tests pass; frontend tests absent
Total	55	51.5	

Strengths

- Full backend feature coverage with strong automated testing
- Clean layered architecture and security controls
- Consistent UI system and responsive implementation

Improvements

- Add frontend test coverage (components/hooks/integration)
- Improve ARIA coverage and run automated a11y checks
- Address deprecation warnings for future compatibility